ENGENHARIA VISUAL DE INTERFACES

Impeccable

Design de Interfaces sem Clichês

O loop de cinco estágios, os seis sub-loops e o piso de qualidade que separa uma tela construída de uma tela montada

OS SEIS NÓS DO LOOP impeccable-design

1. **Avaliar**
2. **Construir**
3. **Melhorar**
4. **Refinar**
5. **Corrigir**
6. **Iterar e Documentar**

Heverton & Ecossistema LegoLoop

07/11/2026 · LegoLoop Foundation

SOBRE ESTA OBRA

Este livro ensina a operar o método Impeccable, criado por Paul Bakaus (upstream v4.7.0), dentro do ecossistema LegoLoop. O Impeccable é um conjunto de cerca de 25 comandos de design de interface organizados em seis sub-loops: Avaliar, Construir, Melhorar, Refinar, Corrigir e Iterar.

A obra segue a progressão Macro, Meso e Micro. A Parte I explica por que interfaces geradas por IA convergem para o mesmo visual genérico e quais pilares o método usa para evitar isso. As Partes II e III descrevem o loop mestre de cinco estágios, seus quatro gates de parada e cada comando dos sub-loops. A Parte IV desce ao nível de implementação: o arquivo loops/impeccable-design.loop.json, o template do Studio Canvas e os agentes especializados. A Parte V traz receitas de uso e a Parte VI reúne glossário, referência rápida e um relatório de conformidade entre o manual e o código.

Fonte primária: docs/manuals/2026-11-01_MANUAL_IMPECCABLE_DESIGN_LOOP.md, complementada pelo arquivo de referência upstream craft-floor.md.

Documento gerado deterministicamente a partir das fontes citadas no próprio texto, via pandoc + typst. Toda afirmação técnica desta obra é rastreável a um arquivo nomeado no capítulo correspondente.

Data da compilação: 07/11/2026 · LegoLoop Foundation

Sumário

ANOTAÇÕES

Como ler este livro 2
A quem este livro se destina 2
A estrutura da obra 2
As convenções da obra 2
PARTE I — Macro: Por que Interfaces de IA Parecem Iguais 3
1. O problema do visual médio 3
2. Os quatro pilares do método 4
3. O que o leitor leva desta parte 5
PARTE II — Meso: Modos de Superfície e o Loop Mestre 6
4. A ambição depende da tela, não do produto 6
5. O loop mestre de cinco estágios 7
6. Por que os gates ficam onde ficam 8
PARTE III — Meso: Os Seis Sub-Loops 10
7. Visão geral da taxonomia 10
8. Sub-loop 1: Avaliar 10
9. Sub-loop 2: Construir e Direcionar 11
10. Sub-loop 3: Melhorar 11
11. Sub-loop 4: Refinar 12
12. Sub-loop 5: Corrigir 12
13. Sub-loop 6: Iterar 13
14. Como escolher o sub-loop certo 13
PARTE IV — Micro: Piso de Qualidade, Agentes e Integração 14
15. O piso de qualidade por dentro 14
16. Os agentes especializados 16
17. Integração no LegoLoop 16
18. Como conferir a integração 17
PARTE V — Prática: Receitas de Uso 19
18. Receita: auditar uma tela existente 19
19. Receita: polir antes de uma entrega 19
20. Receita: redesenhar uma landing page 20
21. Receita: decidir entre duas variantes 20
22. Receita: corrigir um formulário em que usuários erram 21
23. Receita: tornar uma tela mais calma 21
PARTE VI — Apêndices 22
Apêndice A: Glossário 22
Apêndice B: Relatório de conformidade 23
Apêndice C: Referência rápida de comandos 24

Como ler este livro

ANOTAÇÕES

A quem este livro se destina

O livro atende três leitores diferentes, e cada um pode começar por um ponto distinto.

1. **Quem decide sobre produto.** Precisa entender por que uma interface "bonita" gerada por IA ainda parece amadora e como um processo com gates reduz retrabalho. A Parte I responde a isso.
2. **Quem opera o LegoLoop.** Precisa saber qual comando chamar, em que ordem e o que esperar como saída. As Partes II, III e V são o núcleo para esse perfil.
3. **Quem mantém a plataforma.** Precisa saber onde o pipeline está declarado, como o Studio o instancia e onde o manual e o código divergem. A Parte IV e o Apêndice B são escritos para esse leitor.

A estrutura da obra

Parte	Nível	Conteúdo
I	Macro	O problema do visual genérico e os quatro pilares do método
II	Meso	Os quatro modos de superfície e o loop mestre de cinco estágios
III	Meso	Os seis sub-loops e cada um dos seus comandos
IV	Micro	Piso de qualidade, agentes especializados e integração no LegoLoop
V	Prática	Receitas para situações frequentes
VI	Apêndices	Glossário, referência rápida e relatório de conformidade

As convenções da obra

Comandos e caminhos de arquivo aparecem em fonte monoespaçada, como `polish` ou `loops/impeccable-design.loop.json`. Exemplos ilustrativos são marcados como tal no texto. Toda afirmação sobre o LegoLoop é rastreável a um arquivo citado; quando manual e código discordam, o livro segue o código e registra a diferença no Apêndice B.

PARTE I — Macro: Por que Interfaces de IA Parecem Iguais

ANOTAÇÕES

1. O problema do visual médio

Peça a um modelo de linguagem uma "landing page moderna" e o resultado costuma ser previsível: um título enorme sobre um gradiente roxo, um rótulo pequeno em caixa alta acima do título, três cartões do mesmo tamanho com ícone, título e parágrafo, um número grande com legenda pequena e sombras suaves em tudo. Peça de novo, para outro produto, e a página volta quase igual.

Isso não é falta de capacidade do modelo. É o efeito estatístico de como ele foi treinado. Um modelo aprende a produzir o que é mais provável dado o pedido. Para um pedido vago, o mais provável é a média de milhares de páginas parecidas. O resultado é uma interface que não tem erro óbvio, mas também não pertence a nenhum produto em particular.

O manual do Impeccable chama a saída desejada de *out-of-distribution*: uma entrega que sai da média, porque foi decidida e não apenas gerada. O método inteiro existe para trocar o padrão estatístico por decisões explícitas.

Três sintomas que o leitor reconhece

- **A tela montada.** Os componentes estão todos lá, mas nenhum foi pensado para o conteúdo real. Os rótulos dizem "Lorem" ou "Título", os estados de erro não existem e a tela quebra em 375 pixels de largura.
- **O enfeite no lugar da hierarquia.** Gradiente no texto, brilho colorido sem deslocamento, vidro desfocado sem função. Cada um desses recursos tenta compensar a falta de uma hierarquia tipográfica clara.

ANOTAÇÕES

- **A revisão infinita.** O agente tira uma captura de tela, ajusta um detalhe, tira outra, ajusta outro. Sem um critério de parada, o ciclo consome tokens e tempo sem convergir.

> **A DIFERENÇA ENTRE CONSTRUIR E MONTAR**
>
> O arquivo de referência craft-floor.md, que acompanha o upstream do Impeccable, resume a distinção: as superfícies que ninguém desenhou (seleção de texto, cursor, barras de rolagem, anéis de foco, numerais em tabelas) continuam carregando o design. Tematizá-las a partir da paleta é, segundo o próprio documento, o sinal mais barato de que uma página foi construída e não montada, e o que os modelos mais deixam de fazer.

2. Os quatro pilares do método

O manual organiza o Impeccable em quatro pilares. Eles não são etapas; são regras que valem em qualquer etapa.

Pilar 1: o briefing vence

A estética, a paleta, a época e as diretrizes definidas por quem pediu a interface têm prioridade sobre qualquer gosto do agente. O manual é explícito: nunca redirecione uma interface para gosto pessoal arbitrário.

Na prática, isso significa que se o briefing pede uma estética anos 90 com bordas grossas, o agente não "moderniza" a proposta por conta própria. O piso de qualidade, como veremos na Parte IV, trata vários recursos como padrões evitáveis e não como proibições: as palavras do briefing podem autorizar qualquer um deles.

Pilar 2: refinamento não é redesenho

Toda tarefa de interface começa com uma pergunta: estamos refinando ou redesenhando?

Abordagem	O que preserva	O que muda	Quando usar
Refinamento	Identidade, comportamento e texto	Alinhamento, ritmo, consistência	A tela funciona, mas está mal acabada
Redesenho	Apenas o conteúdo e a função	Todo o mundo visual	A tela atual é anti-referência

ANOTAÇÕES

No redesenho, a interface antiga vira evidência do que não fazer. O novo mundo visual precisa ser documentado, e não apenas aplicado, para que a próxima sessão o reconheça.

Pilar 3: o piso de qualidade é inegociável

Antes de qualquer edição de UI, inclusive ajustes pontuais, a leitura do arquivo `craft-floor.md` é obrigatória. O manual cita alguns exemplos do que esse piso proíbe: gradientes sem contraste, bordas excessivas, tipografia desordenada e emojis na interface. A Parte IV dedica um capítulo inteiro ao conteúdo desse arquivo.

Pilar 4: verificação delimitada

O quarto pilar ataca diretamente o sintoma da revisão infinita. A regra tem três partes:

1. Inspecione em lote: desktop e mobile na mesma rodada.
2. Corrija todos os apontamentos em um único bloco de edição.
3. Finalize com, no máximo, mais uma checagem.

> **Uma rodada de captura, um bloco de correções, uma checagem final. Não existe quarta volta.**

O nome em inglês, *bounded pass*, descreve bem a ideia: a verificação tem uma fronteira conhecida antes de começar. Isso torna o custo previsível e força o agente a olhar para todos os problemas de uma vez, em vez de corrigi-los um por um à medida que aparecem.

3. O que o leitor leva desta parte

O Impeccable não é uma biblioteca de componentes nem um tema visual. É um processo com quatro regras permanentes: respeitar o briefing, decidir entre refinar e redesenhar, cumprir o piso de qualidade e limitar a verificação. As próximas partes mostram como essas regras viram estágios, gates e comandos concretos.

PARTE II — Meso: Modos de Superfície e o Loop Mestre

ANOTAÇÕES

4. A ambição depende da tela, não do produto

Um mesmo produto tem telas com objetivos muito diferentes. A página de preços precisa convencer; o painel de configurações precisa ser rápido de usar; a documentação precisa ser lida sem cansaço. O Impeccable mede a ambição visual pelo **modo da superfície**, e não pelo produto como um todo.

Modo	O visitante precisa	Exemplos	Prioridade de design
Operate	Completar uma tarefa	Dashboards, admin, editores	Escaneabilidade e contenção
Persuade	Decidir e agir	Landing pages, preços	Hierarquia de valor e conversão
Read	Compreender um conteúdo	Docs, manuais, changelogs	Ritmo de leitura sem distração
Experience	Imergir na própria obra	Portfólios, galerias	A interface recua, a obra lidera

Como o modo muda as decisões

No modo **Operate**, a ambição aparece como precisão: densidade coerente, alinhamentos exatos, comportamentos que o usuário já espera da plataforma. Um painel administrativo com tipografia expressiva e animações de entrada em cada seção está no modo errado.

No modo **Persuade**, a tipografia pode ser expressiva e a estética pode ser imersiva, porque o objetivo é mover o visitante para uma decisão. É o único modo, ao lado de Experience, em que o comando `overdrive` é permitido.

No modo **Read**, o contraste é descansado e a hierarquia tipográfica faz quase todo o trabalho. O piso de qualidade fixa a medida do corpo de texto entre 65 e 75 caracteres por linha.

No modo **Experience**, a interface sai do caminho. Desde a primeira dobra, quem lidera é a obra exibida: a fotografia, o projeto, a peça interativa.

ANOTAÇÕES

REGRA DE CLASSIFICAÇÃO

Classifique cada tela isoladamente. Um produto SaaS típico tem a landing em Persuade, o app em Operate e a central de ajuda em Read. Aplicar a mesma ambição às três é o erro mais comum de quem começa no método.

5. O loop mestre de cinco estágios

O fluxo completo do Impeccable atravessa cinco estágios. Entre eles existem quatro gates binários de parada obrigatória, chamados P1 a P4. Binário quer dizer que o gate só tem duas respostas: passa ou não passa.

1. Contexto e Auditoria PRODUCT.md, DESIGN.md, critique, audit

↓ **P1: achados e severidade**

2. Direção Visual classificar modo, shape, preservar ou substituir

↓ **P2: direção visual aprovada**

3. Execução Cirúrgica craft-floor.md, layout, typeset, colorize, clarify

↓ **P3: antes e depois por item**

4. Verificação em Lote captura mobile 375px e desktop, polish, revisão

↓ **P4: veredito VERIFIED**

5. Documentação e Entrega document, extract, proposta de PR

Estágio 1: Contexto e Auditoria

O agente carrega os dois documentos de contexto durável, `PRODUCT.md` e `DESIGN.md`, e então roda duas avaliações: `critique`, uma revisão heurística de experiência do usuário, e `audit`, uma checagem técnica de acessibilidade, performance e viewports.

Gate P1. O estágio só termina quando os achados estão listados com severidade. Sem severidade, não há como decidir o que corrigir primeiro.

Estágio 2: Direção Visual

Com o diagnóstico em mãos, o agente classifica o modo da superfície, planeja a experiência antes do código com `shape` e decide o destino do trabalho novo: preservar, expandir ou substituir o que existe.

ANOTAÇÕES

Gate P2. A direção visual precisa de aprovação humana explícita. É o gate mais barato do loop: errar a direção aqui custa uma conversa; errar depois custa uma reimplementação.

Estágio 3: Execução Cirúrgica

A primeira ação do estágio é a leitura obrigatória de `craft-floor.md`. Depois vêm, nesta ordem, espaçamento e ritmo (`layout`), hierarquia tipográfica (`typeset`), paleta estratégica (`colorize`) e, por fim, textos e casos de borda (`harden` e `clarify`).

A ordem não é arbitrária. Cor aplicada antes de a hierarquia estar resolvida tende a ser usada como muleta para criar destaque que a tipografia deveria criar.

Gate P3. Cada item alterado precisa de um antes e depois. Isso torna a revisão do gate objetiva: o revisor compara pares, não opina sobre o todo.

Estágio 4: Verificação em Lote

Aqui o pilar da verificação delimitada vira prática. Uma captura em lote cobre mobile em 375 pixels e desktop. O comando `polish` faz o acabamento fino e o agente revisor de acabamento julga o resultado.

Gate P4. O revisor emite um veredito. Só `VERIFIED` libera o estágio seguinte.

Estágio 5: Documentação e Entrega

O comando `document` atualiza o `DESIGN.md` a partir do código real, e não das intenções. O comando `extract` isola tokens e componentes para o design system. O estágio termina com uma proposta de entrega ou um pull request.

> **O DESIGN.md descreve o que foi entregue, não o que se pretendia entregar.**

6. Por que os gates ficam onde ficam

Cada gate fica no ponto em que o próximo estágio ficaria caro se começasse com uma premissa errada.

Gate	Fica antes de	Custo evitado
P1	Decidir a direção	Direção escolhida sem diagnóstico
P2	Escrever código	Reimplementar uma tela inteira
P3	Verificar em lote	Revisão subjetiva sem comparação
P4	Documentar	Registrar no DESIGN.md algo reprovado

ANOTAÇÕES

PARTE III — Meso: Os Seis Sub--Loops

ANOTAÇÕES

7. Visão geral da taxonomia

Os cerca de 25 comandos do Impeccable estão agrupados em seis sub--loops reutilizáveis. Cada sub-loop tem um objetivo único, o que permite chamá-lo isoladamente quando a tarefa não exige o fluxo completo.

Sub--loop	Nome	Objetivo em uma frase
1	Avaliar	Radiografar o estado atual sem alterar nada
2	Construir e Direcionar	Fixar contratos de produto antes de codificar
3	Melhorar	Elevar a qualidade pelos fundamentos visuais
4	Refinar	Preparar a interface para produção
5	Corrigir	Resolver fricções específicas
6	Iterar	Explorar alternativas visuais ao vivo

8. Sub-loop 1: Avaliar

O objetivo é diagnosticar sem fazer alterações prematuras. Um agente que corrige enquanto avalia perde a visão do conjunto e tende a atacar o primeiro problema que encontra, não o mais grave.

- `critique` faz a revisão heurística: usabilidade, hierarquia de informação e carga cognitiva. Responde à pergunta "o usuário entende o que fazer?".
- `audit` faz a auditoria técnica: acessibilidade segundo a WCAG, performance de renderização e responsividade em vários viewports. Responde à pergunta "a tela funciona para todos?".

Os dois comandos são complementares. Uma tela pode passar no `audit` com contraste correto e foco visível e ainda assim falhar no `critique` por esconder a ação principal.

ANOTAÇÕES

9. Sub-loop 2: Construir e Direcionar

Este sub-loop estabelece as fundações conceituais e os contratos duráveis de produto antes de qualquer código.

- `init` captura o contexto durável da aplicação em `PRODUCT.md`: para quem é, o que resolve, em que tom fala.
- `shape` planeja fluxos de tela e interações sem escrever código.
- `document` gera o `DESIGN.md` a partir da verdade do código.
- `extract` garimpa tokens e componentes consolidados para o design system.

> **DOIS DOCUMENTOS, DOIS PAPÉIS**
>
> PRODUCT.md responde "o quê e para quem". DESIGN.md responde "como está construído". O primeiro é escrito antes do código, a partir de decisões. O segundo é escrito depois, a partir do código entregue. Inverter essa ordem produz um DESIGN.md cheio de intenções que o código não cumpre.

10. Sub-loop 3: Melhorar

O objetivo é elevar a qualidade sensorial e tátil da interface pelos fundamentos visuais. São seis comandos, e a ordem em que aparecem é a ordem recomendada.

Comando	Atua sobre	Pergunta que responde
`layout`	Respiros, grids, alinhamento, ritmo	O olho encontra o caminho?
`typeset`	Proporção, peso, escala tipográfica	A hierarquia é óbvia sem cor?
`colorize`	Paleta com significado semântico	Cada cor quer dizer algo?
`animate`	Micro-interações funcionais	O movimento explica uma mudança?
`delight`	Detalhes memoráveis e estados táteis	Há um momento que se lembra?
`overdrive`	Expressividade estética ampliada	Só em Persuade ou Experience

O comando `colorize` evita dois extremos que o manual nomeia: o monocromatismo insosso e o carnaval de cores. A cor só entra quando carrega significado: estado, categoria, ação primária.

ANOTAÇÕES

O comando `overdrive` merece atenção especial. Ele é reservado exclusivamente aos modos Persuade e Experience. Usá-lo em uma tela Operate é um erro de classificação, não uma escolha estética.

11. Sub-loop 4: Refinar

Este sub-loop blinda a interface para ambientes de missão crítica.

- `polish` é o passe de micropolimento e consistência antes da entrega.
- `harden` trata estados de erro, carregamento, internacionalização e limites de dados: o nome com 80 caracteres, a lista vazia, a lista com dez mil itens, a tradução alemã que dobra o tamanho do botão.
- `onboard` cria os fluxos de primeira execução e os estados vazios.
- `distill` remove complexidade visual desnecessária e elementos ruidosos.
- `bolder` e `quieter` calibram o tom visual: o primeiro aumenta contraste, o segundo reduz estimulação excessiva.

> **BOLDER E QUIETER NÃO SÃO OPOSTOS DE GOSTO**
>
> Os dois comandos corrigem um desvio em relação ao modo da superfície. Uma tela Persuade sem energia pede bolder. Uma tela Operate que grita pede quieter. A decisão vem do modo, não da preferência de quem revisa.

12. Sub-loop 5: Corrigir

O objetivo é resolver fricções ergonômicas e falhas operacionais pontuais.

- `clarify` refatora a UX copy: microtexto, rótulos de botões e mensagens de validação. O piso de qualidade dá o critério: controles nomeiam sua ação; erros nomeiam o problema e a forma de recuperação.
- `adapt` adequa a tela a extremos: mobile compacto, ultra-wide e layouts nativos.
- `optimize` diagnostica e corrige gargalos de renderização na árvore de componentes.

ANOTAÇÕES

Um botão que diz "Enviar" está correto, mas um botão que diz "Publicar artigo" está melhor, porque nomeia a consequência. Uma mensagem "Erro inválido" falha nos dois critérios: não diz qual é o problema nem como sair dele.

13. Sub-loop 6: Iterar

O último sub-loop serve para explorar alternativas visuais em tempo real.

- `live` permite inspecionar e selecionar elementos diretamente no navegador para gerar variantes ao vivo.
- `generate` cria variações simultâneas de um elemento no navegador para uma decisão imediata.

Iterar é útil quando a direção está aprovada, mas um elemento específico ainda não convence. Em vez de três rodadas de descrição verbal, o agente mostra três versões lado a lado.

14. Como escolher o sub-loop certo

Situação	Sub-loop	Comando de entrada
Não sei o que está errado	Avaliar	`critique`
Tela nova, sem código ainda	Construir	`shape`
Funciona, mas parece amadora	Melhorar	`layout`
Vai para produção amanhã	Refinar	`harden`
Usuários erram um formulário	Corrigir	`clarify`
Duas opções e nenhuma decisão	Iterar	`generate`

PARTE IV — Micro: Piso de Qualidade, Agentes e Integração

ANOTAÇÕES

15. O piso de qualidade por dentro

O arquivo `craft-floor.md` fica em `upstream/reference/` dentro da skill `aidd-impeccable`. Ele é carregado depois que a direção está fixada e tem duas seções: o que verificar e o que recusar. O próprio arquivo avisa que um briefing fixado ou o mundo visual aprovado tem prioridade sobre qualquer regra dele; o hábito do agente, não.

O que verificar

Cada item é uma checagem sobre o resultado renderizado, e não uma intenção. Todos são conferidos na mesma captura.

Eixo	Critério verificável
Contraste	Corpo e placeholder ≥ 4,5:1; texto grande ≥ 3:1
Profundidade	Sombras com deslocamento e desfoque suave
Espaçamento	Grupos apertados, separação generosa
Tipografia	Corpo entre 65 e 75ch; display até 6rem
Tracking	Piso de −0,04em; −0,02 a −0,03em lê melhor
Movimento	Um momento autoral, ease-out exponencial
Estados	Hover, disabled, loading, error e empty
Superfícies	Seleção, cursor, scrollbar e foco tematizados
Texto	Linguagem do produto; erros com saída
Cobertura	Todo requisito do briefing achável em segundos

Sobre o contraste em superfícies coloridas, o arquivo acrescenta uma regra fina: o texto secundário deve ser tingido a partir do matiz da superfície ou do primeiro plano, nunca cinza.

O que recusar

O arquivo trata a maior parte desta lista como padrões da categoria, e não como proibições: o briefing pode autorizar qualquer um deles. Recorrer a um deles quando o eixo está livre, diz o texto, significa que o agente não estava decidindo.

Estruturas de página:

ANOTAÇÕES

- Cartões do mesmo tamanho com ícone, título e texto como estrutura da página. Cartões aninhados são sempre errados.
- O modelo de métrica de destaque: número grande, legenda pequena, estatísticas de apoio.
- Números de seção como 01, 02, 03, a menos que a sequência carregue informação.
- Um modal para uma tarefa que não precisa de interrupção nem de foco protegido.

Hábitos de superfície:

- Texto com gradiente. A ênfase vem de peso ou tamanho.
- Vidro e desfoque como decoração.
- Borda lateral colorida acima de 1px em cartões, itens de lista ou alertas.
- Sombras duras deslocadas fora de um mundo de fato neobrutalista.
- Fonte monoespaçada como fantasia de "técnico", e não para código ou dados.
- Glifos Unicode ou emoji no lugar de um sistema de ícones.
- Claro ou escuro escolhido pela categoria, e não pela cena de uso.

> **A ÚNICA PROIBIÇÃO ABSOLUTA**
>
> Entre todos os itens, o arquivo marca um como banimento, e não como padrão: o rótulo pequeno (kicker ou eyebrow) acima de um título. Nenhum briefing o recupera. O título carrega o próprio peso; o rótulo deve ser apagado.

O arquivo também fixa limites numéricos: raio de cartões entre 12 e 16 pixels, com formato de pílula reservado a controles pequenos, e elevação declarada uma única vez, por borda ou por sombra. Uma borda de 1px sob uma sombra larga e suave é chamada de "cartão fantasma".

Por fim, uma regra de honestidade que vale para qualquer interface: afirmações e configurações vêm de dados fornecidos, e valores ilustrativos são rotulados como tal.

ANOTAÇÕES

16. Os agentes especializados

Para evitar contaminação de contexto, o ecossistema do Impeccable delega quatro papéis a agentes autônomos. Cada um recebe apenas o que precisa para sua função.

Agente	Papel	Momento
Documenter	Gera DESIGN.md a partir do código entregue	Estágio 5
Finish reviewer	Julga o acabamento contra a direção	Gate P4
Asset producer	Produz ícones e vetores SVG inline	Estágio 3
Manual edit applier	Aplica micro-ajustes da auditoria	Estágios 3 e 4

O manual descreve o revisor de acabamento como um diretor de arte intransigente: ele decide se o resultado alcançou o nível esperado ou precisa ser refeito. Os SVG do produtor de ativos seguem um padrão semântico, sem emoji e com `stroke="currentColor"`, para herdarem a cor do texto ao redor.

A separação em agentes tem uma razão prática. O agente que implementou uma tela tende a defendê-la. Um revisor que só recebe a direção aprovada e o resultado final julga sem esse viés.

17. Integração no LegoLoop

No LegoLoop, o pipeline existe em duas formas canônicas, ambas verificáveis no repositório.

A peça de loop no acervo

O arquivo `loops/impeccable-design.loop.json` segue o esquema `dag.v1` e se chama "Pipeline de Design Impeccable (6 Sub-Loops)", versão 1.0.0. Ele declara seis nós em cadeia linear, cada um dependente do anterior, com o artefato exigido e o modelo sugerido em `routing`:

Nó	Artefato que o gate exige	Modelo sugerido
`subloop-avaliar`	`docs/reports/AUDITORIA_DESIGN.md`	sonnet, ou haiku
`subloop-construir`	`docs/specs/DIRECAO_VISUAL.md`	sonnet, ou haiku
`subloop-melhorar`	`src/styles/theme.css`	não declarado
`subloop-refinar`	`docs/reports/HARDENING_UI.md`	não declarado
`subloop-corrigir`	`docs/reports/RESPONSIVE_VERIFICATION.md`	não declarado
`subloop-iterar-documentar`	`DESIGN.md`	não declarado

ANOTAÇÕES

Cada nó tem um gate mecânico do tipo `file_exists(...)`: o nó só é considerado concluído se o artefato existir no disco. É um gate de existência, não de qualidade; quem julga o conteúdo continua sendo o protocolo humano P1 a P4 conduzido pela skill.

Todos os seis nós apontam para a mesma peça, `skills/impeccable`. O que muda entre eles é o sub-loop que a peça executa.

O template do Studio Canvas

Em `internal/studio/templates.go`, o template "Pipeline de Design Impeccable (6 Sub-Loops)" aparece no menu de templates rápidos com o ícone `palette`. Ao instanciá-lo, o canvas recebe a esteira completa com o roteamento cognitivo pré-configurado.

> **DO ARQUIVO AO CANVAS**
>
> O JSON do acervo e o template do Studio descrevem a mesma esteira por dois caminhos: o primeiro é executado pelo motor DAG via CLI; o segundo é montado visualmente para quem prefere arrastar nós. Alterar um não altera o outro automaticamente, e por isso vale conferir os dois quando o pipeline evoluir.

A skill no harness

A skill `aidd-impeccable` funciona como maestro: recebe um pedido em linguagem natural, como "auditar a tela dashboard" ou "polir antes de entregar", e escolhe comando, modo e ordem. Ela carrega a metodologia de uma cópia local do upstream, sem depender de plugin, e por isso roda em qualquer harness que leia arquivos.

18. Como conferir a integração

Três comandos do CLI `legoloop` bastam para confirmar que a esteira está íntegra antes de usá-la.

```
# 1. Valida contrato, gates e ciclos do DAG
legoloop validate loops/impeccable-design.loop.json
```

ANOTAÇÕES

```
# 2. Executa a esteira pelo motor DAG
legoloop run loops/impeccable-design.loop.json

# 3. Abre o Looping Studio (padrão: 127.0.0.1:7878)
legoloop studio
```

Em 07/11/2026, o primeiro comando respondeu `6 nós válidos, sem ciclos`, em seis camadas de um nó cada. Uma camada reúne nós que podem rodar ao mesmo tempo; como cada sub-loop depende do anterior, uma camada com dois nós aqui seria erro de declaração.

PARTE V — Prática: Receitas de Uso

ANOTAÇÕES

As receitas a seguir combinam sub-loops para situações frequentes. São roteiros ilustrativos montados a partir do manual; os nomes de tela e os números servem de exemplo e não descrevem um projeto real.

18. Receita: auditar uma tela existente

Quando usar. Alguém diz que "o dashboard está estranho", mas ninguém sabe dizer o quê.

1. Classifique o modo. Um dashboard é quase sempre Operate.
2. Rode `critique` e `audit` em sequência, sem corrigir nada.
3. Consolide os achados em uma lista com severidade alta, média e baixa.
4. Apresente a lista e pare. Este é o gate P1.

Saída esperada. Uma lista como esta, ilustrativa:

Severidade	Achado	Origem
Alta	Ação principal abaixo da dobra em 375px	`critique`
Alta	Contraste de 3,1:1 no texto secundário	`audit`
Média	Quatro tamanhos de título sem escala clara	`critique`
Baixa	Scrollbar com o estilo padrão do navegador	`audit`

Erro comum. Corrigir o contraste enquanto ainda está avaliando. A correção isolada parece produtiva, mas impede de ver que o problema de hierarquia talvez peça uma reorganização maior.

19. Receita: polir antes de uma entrega

Quando usar. A tela está aprovada e funcionando; falta acabamento.

1. Leia `craft-floor.md`.
2. Rode `harden` para cobrir estados de erro, vazio e carregamento.
3. Rode `clarify` sobre todos os rótulos e mensagens.
4. Faça uma captura em lote, mobile e desktop.
5. Rode `polish` e corrija tudo em um único bloco.

6. Faça no máximo uma checagem final.

ANOTAÇÕES

> **O LIMITE DA RECEITA**
>
> Se a checagem final encontrar um problema estrutural, como a hierarquia que não funciona no mobile, a receita de polimento não é o lugar para resolvê-lo. Volte ao sub-loop Melhorar com layout e typeset, e só depois retome o polimento.

20. Receita: redesenhar uma landing page

Quando usar. A página atual é anti-referência: ninguém quer preservar nada dela além do conteúdo.

1. Rode `init` se não existir `PRODUCT.md`.
2. Use a página antiga apenas como evidência: liste o que ela faz mal.
3. Classifique o modo como Persuade.
4. Rode `shape` para planejar o fluxo sem código e peça aprovação da direção (P2).
5. Execute `layout`, `typeset` e `colorize`, nesta ordem.
6. Considere `overdrive`, que só é permitido aqui por causa do modo.
7. Verifique em lote (P4) e rode `document` para registrar o novo mundo visual.

21. Receita: decidir entre duas variantes

Quando usar. A direção está aprovada, mas o bloco de preços tem duas propostas e a equipe não decide.

1. Rode `generate` sobre o bloco para gerar as variações lado a lado no navegador.
2. Use `live` para ajustar um detalhe da variante preferida.
3. Registre a escolha e o motivo.
4. Volte ao estágio 4 com a variante escolhida.

ANOTAÇÕES

22. Receita: corrigir um formulário em que usuários erram

Quando usar. O formulário de cadastro tem alta taxa de abandono e o suporte recebe perguntas sobre as mesmas mensagens de erro.

1. Rode `critique` focado no formulário para localizar onde o usuário hesita.
2. Rode `clarify` sobre rótulos, textos de ajuda e mensagens de validação.
3. Rode `harden` para cobrir os limites: nome muito longo, campo vazio, colagem de texto com espaços.
4. Faça a captura em lote, mobile e desktop, com os estados de erro visíveis.

O critério do `clarify` vem do piso de qualidade: cada erro precisa nomear o problema e a forma de recuperação. A tabela abaixo mostra o tipo de troca esperada; as mensagens são ilustrativas.

Antes	Depois
Erro inválido	O CPF precisa ter 11 números. Confira os dígitos.
Campo obrigatório	Informe um e-mail para receber a confirmação.
Senha fraca	Use 8 caracteres ou mais, com pelo menos um número.
Enviar	Criar conta

23. Receita: tornar uma tela mais calma

Quando usar. Usuários de um painel interno reclamam de cansaço visual.

1. Confirme o modo Operate.
2. Rode `distill` para remover elementos que não ajudam a tarefa.
3. Rode `quieter` para reduzir a estimulação.
4. Revise com `audit` para garantir que o contraste mínimo continua respeitado depois da redução.

O passo 4 existe porque reduzir estímulo visual costuma significar clarear cores, e cores claras demais derrubam o contraste abaixo de 4,5:1.

PARTE VI — Apêndices

ANOTAÇÕES

Apêndice A: Glossário

Bounded pass (verificação delimitada). Rodada de revisão com fronteira fixada antes de começar: uma captura em lote, um bloco de correções, no máximo mais uma checagem.

Briefing. Conjunto de diretrizes de quem pediu a interface: estética, paleta, época, tom. Tem prioridade sobre o gosto do agente.

Cartão fantasma. Cartão com borda de 1px sob uma sombra larga e suave; declara elevação duas vezes.

craft-floor.md. Arquivo de referência do upstream que define o piso de qualidade: o que verificar e o que recusar.

DESIGN.md. Especificação de design gerada a partir do código entregue.

Gate binário. Ponto de parada com apenas duas saídas: passa ou não passa.

Modo de superfície. Classificação de uma tela por objetivo: Operate, Persuade, Read ou Experience.

Out-of-distribution. Resultado que sai da média estatística do que um modelo produziria por padrão.

PRODUCT.md. Contexto durável da aplicação: público, problema e tom.

Refinamento. Melhoria que preserva identidade, comportamento e texto.

Redesenho. Novo mundo visual em que a interface antiga serve apenas de evidência.

Sub-loop. Grupo de comandos com um objetivo único, reutilizável de forma isolada.

Apêndice B: Relatório de conformidade

ANOTAÇÕES

Este apêndice compara o manual de 01/11/2026 com o estado do repositório em 07/11/2026. O objetivo é que o leitor saiba exatamente onde o texto do manual e o código coincidem.

Ponto verificado	Manual	Código	Situação
Arquivo do loop	`impeccable-design.loop.json`	Existe em `loops/`	Conforme
Número de nós	6 sub-loops	6 nós lineares	Conforme
Sexto nó	Iterar	`subloop-iterar-documentar`	Diverge no nome
Peça executora	Não especificada	`skills/impeccable` em todos	Informativo
Template do Studio	Ícone `palette`	Ícone `palette`	Conforme
Nome do template	Não especificado	"...(6 Sub-Loops)"	Informativo
Estágios e gates P1-P4	5 estágios, 4 gates	Gates `file_exists` por nó	Diverge
Nome do revisor	`impeccabe-finish-reviewer`	`impeccable-finish-reviewer`	Erro de grafia

Leitura do relatório. O DAG declara apenas gates de existência de arquivo por nó; os gates de julgamento P1 a P4 são protocolo da skill, conduzido no chat. No sexto nó, o código agrupa iteração e documentação, enquanto o manual trata `document` como comando do sub-loop Construir. Para o revisor, o livro usa a grafia do agente instalado.

Apêndice C: Referência rápida de comandos

ANOTAÇÕES

Comando	Sub-loop	Função resumida
`critique`	Avaliar	Revisão heurística de UX
`audit`	Avaliar	Acessibilidade, performance, viewports
`init`	Construir	Cria PRODUCT.md
`shape`	Construir	Planeja fluxos sem código
`document`	Construir	Gera DESIGN.md do código
`extract`	Construir	Isola tokens e componentes
`layout`	Melhorar	Espaçamento, grid, ritmo
`typeset`	Melhorar	Escala e pesos tipográficos
`colorize`	Melhorar	Paleta com significado
`animate`	Melhorar	Micro-interações funcionais
`delight`	Melhorar	Detalhes memoráveis
`overdrive`	Melhorar	Expressividade; só Persuade e Experience
`polish`	Refinar	Micropolimento final
`harden`	Refinar	Erros, loading, i18n, limites
`onboard`	Refinar	Primeira execução e estados vazios
`distill`	Refinar	Remove ruído visual
`bolder`	Refinar	Aumenta contraste e presença
`quieter`	Refinar	Reduz estimulação
`clarify`	Corrigir	UX copy e mensagens
`adapt`	Corrigir	Telas extremas
`optimize`	Corrigir	Gargalos de renderização
`live`	Iterar	Variantes ao vivo no navegador
`generate`	Iterar	Variações simultâneas